

Análisis Tecnológico FULL: Arquitectura de Amazon y Escalabilidad en AWS

Para el proyecto G502

1 de noviembre de 2025

Introducción

La tienda Amazon.com es el ejemplo canónico de una arquitectura de **Microservicios a Escala Web**. Su evolución técnica interna es la razón fundamental por la que se creó y lanzó **Amazon Web Services (AWS)**, la infraestructura en la nube más grande del mundo.

1. I. Fundamento de la Arquitectura: De Monolito a Microservicios

La arquitectura de Amazon se basa en la división de su sistema monolítico original en miles de servicios autónomos.

1.1. Principios de Diseño Clave

1. Arquitectura de Microservicios:

$$\text{Amazon} = \sum_{i=1}^N \text{Microservicio}_i \quad \text{donde } N \gg 1000$$

Cada servicio es propiedad de un equipo pequeño (*Two-Pizza Team*) y se implementa de forma autónoma. Esto garantiza **Agilidad y Desacoplamiento**: un fallo en un servicio menor (ej. $S_{\text{reseñas}}$) no afecta la función crítica de caja (S_{checkout}).

2. Bases de Datos Políglotas: Cada microservicio elige la base de datos más adecuada, siguiendo el principio:

$$\text{Persistencia Óptima} \propto \frac{1}{\text{Tipo de Dato}}$$

- **Catálogo y Sesiones (NoSQL, Baja Latencia):** Amazon DynamoDB.
- **Transacciones Financieras (SQL, Atomicidad):** PostgreSQL o Amazon Aurora.
- **Relaciones/Recomendaciones (Grafo):** Amazon Neptune.

3. Arquitectura Basada en Eventos (EDA): La comunicación es principalmente **asíncrona** a través de colocación de mensajes, logrando **Resiliencia Extrema**.

$$\text{Acción} \xrightarrow{\text{Publicar}} \text{Cola/Tópico} \xrightarrow{\text{Consumir}} \text{Reacción}$$

- **Servicios:** Amazon SQS (colas de mensajes simples) y Amazon SNS (notificaciones).

4. Stack Políglota de Lenguajes: Se permite la **Libertad Tecnológica**. Utilizan Java (histórico), Python (para ML/data), y Go/Rust (para servicios de rendimiento crítico). En el Frontend, utilizan frameworks modernos como React y JavaScript/TypeScript.

2. II. Capa de Servicios Centrales (AWS Core)

La tienda utiliza los siguientes servicios de su propia nube para las funciones esenciales:

Servicio AWS	Función Específica en Amazon.com	Concepto
Amazon S3	Almacenamiento de Objetos para imágenes, videos, logs y copias de seguridad. Garantiza durabilidad (99,999999999 %).	O_{Storage}
Amazon EC2	Proporciona las máquinas virtuales para ejecutar los microservicios más grandes. Permite el escalado horizontal .	C_{Compute}
AWS Lambda	Ejecuta código de <i>backend</i> en respuesta a eventos (serverless). No requiere gestión de servidor.	$C_{\text{Serverless}}$
CloudFront	Red de Entrega de Contenido (CDN) Global . Almacena en caché el contenido cerca del usuario, reduciendo la Latencia (τ).	N_{CDN}
ELB	Distribuye el tráfico de entrada entre múltiples instancias EC2 en diferentes Zonas de Disponibilidad.	$N_{\text{LoadBalancer}}$
Kinesis	Ingesta y procesa flujos masivos de datos de clics y navegación en tiempo real, alimentando la personalización.	$D_{\text{Streaming}}$

3. III. Inteligencia Artificial y Logística

Aquí es donde Amazon convierte los datos en ventaja competitiva.

1. **Recomendaciones (ML/AI):** Utilización de **Amazon Personalize** (o modelos internos entrenados con **SageMaker**) para calcular la **Propensión a Comprar** (P_{compra}) y generar sugerencias altamente personalizadas.
2. **Búsqueda (Relevancia):** Algoritmos de *Machine Learning* clasifican los productos no solo por palabras clave, sino por su probabilidad de ser comprados por el usuario.
3. **Logística Inteligente (Fulfillment):** Sistemas de gestión de almacén con *Computer Vision* e IA que optimizan rutas de robots y embalaje para la **entrega predictiva**.

4. IV. La Arquitectura Altamente Escalable en AWS (Principios)

La Escalabilidad no es un producto, sino una metodología de diseño fundamental para manejar millones de usuarios y peticiones (Peticiones $\gg 10^6$).

4.1. 1. Principio de Elasticidad y Escalamiento Horizontal

El sistema se escala añadiendo más recursos pequeños, no mediante un único recurso más grande.

$$\text{Escalamiento Horizontal} \propto \frac{1}{\text{Escalamiento Vertical}}$$

- **Auto Scaling:** Ajusta el número de servidores $N_{\text{instancias}}$ en función de la demanda y el uso de CPU.

$$N_{\text{instancias}}(t) \leftarrow \begin{cases} N_{\text{instancias}}(t-1) + \Delta N & \text{si } \text{UsocPU} > \theta_{\text{Alto}} \\ N_{\text{instancias}}(t-1) - \Delta N & \text{si } \text{UsocPU} < \theta_{\text{Bajo}} \end{cases}$$

- **Serverless:** AWS Lambda y API Gateway eliminan la gestión de capacidad, escalando la infraestructura automáticamente al código.

4.2. 2. Principio de Resiliencia y Alta Disponibilidad

Se prioriza la tolerancia a fallos sobre la prevención de fallos, maximizando la Disponibilidad (A):

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}}$$

- **Zonas de Disponibilidad (AZs):** El despliegue se realiza en $N_{AZs} \geq 3$ centros de datos aislados físicamente.
- **Replicación de Datos:** Uso de `Amazon Aurora` y `Read Replicas` para replicar bases de datos en diferentes AZs, permitiendo una conmutación por error instantánea.
- **Protección:** `AWS WAF` (Web Application Firewall) y `AWS Shield` protegen la capa de aplicación y red de ataques DDoS.

4.3. 3. Principio de Observabilidad (Monitoreo)

Es esencial para la gestión y la optimización de costes.

- **Monitoreo:** `Amazon CloudWatch` se utiliza para recolectar métricas de rendimiento ($M = \{\text{CPU}, \tau_{DB}, \text{Tasa de Error}\}$).
- **Auditoría y Compliance:** `AWS CloudTrail` registra todas las acciones de la API, esencial para la seguridad y la trazabilidad.
- **Alertas Proactivas:** Configuración de alarmas para notificar *antes* de que ocurra un fallo.

Conclusión para G502

La implementación de estos principios de Microservicios, Bases de Datos Políglotas y Resiliencia de AWS es la hoja de ruta de clase mundial para lograr la escalabilidad deseada en **g502**.